

DBMS – PRODUCT REVIEW AND RATING MANAGEMENT

Project Title: IKEA E-Commerce Database Management System

1. Database Design

1.1 Review Table

Attribute	Data Type	Key	Description
Review_ID	INT	PK	Unique review ID
Customer_ID	INT	FK	Customer who submitted the review
Product_ID	INT	FK	Product being reviewed
Review_Text	VARCHAR(500)	-	Customer feedback
Review_Date	DATE	-	Date of review

1.2 Rating Table

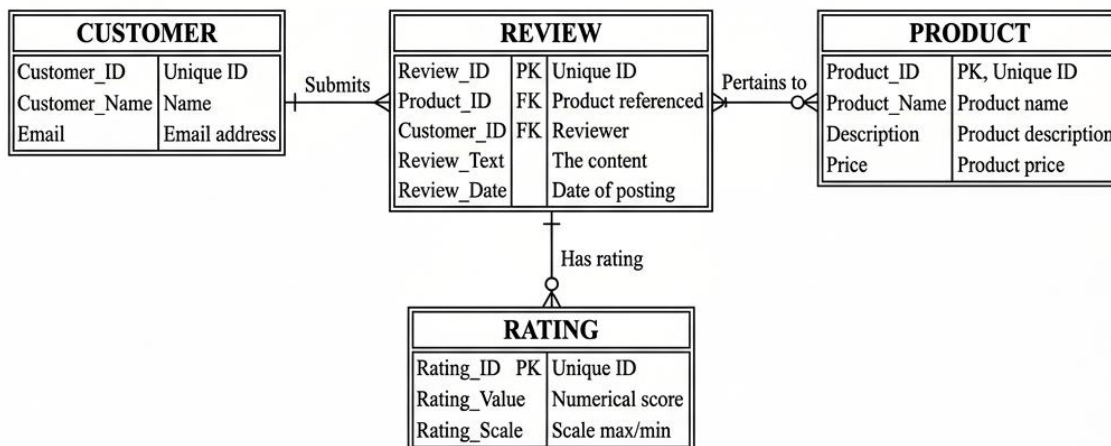
Attribute	Data Type	Key	Description
Rating_ID	INT	PK	Unique rating ID
Review_ID	INT	FK	References Review
Rating_Value	INT	-	Rating given from 1 to 5
Rating_Date	DATE	-	Date of rating

2. Table Relationships

FIGURE 1: E-Commerce Database Relationships (Customer and Product)

2. TABLE RELATIONSHIPS:

2.1 SCHEMA DESIGN:



2.2 RELATIONSHIP SUMMARY:

CUSTOMER - REVIEW RELATIONSHIP (1:M)	
Attributes	Description
Customer_ID	Found in both tables. Links customer to review. Each review belongs to exactly one customer, but one customer can write many reviews.
PRODUCT - REVIEW RELATIONSHIP (1:M)	
Attributes	Description
Product_ID	Found in both tables. Links review to product. Each review is about one product, but one product can have many reviews.
REVIEW - RATING RELATIONSHIP (1:M)	
Attributes	Description
Rating_ID	Found in both tables. A review can generate a detailed numerical rating structure, or be linked to a rating matrix. One review can contain multiple individual ratings (e.g., quality, usability, shipping speed).

One customer can submit multiple reviews, while each review belongs to one customer. A product can receive multiple reviews, while each review is associated with one product. Each review has one corresponding rating.

3. Table Creation

3.1 Create Review Table

```
SQL> CREATE TABLE Review (  
2     Review_ID INT PRIMARY KEY,  
3     Customer_ID INT,  
4     Product_ID INT,  
5     Review_Text VARCHAR2(500),  
6     Review_Date DATE  
7 );
```

Table created.

3.2 Create Rating Table

```
SQL> CREATE TABLE Rating (  
2     Rating_ID INT PRIMARY KEY,  
3     Review_ID INT,  
4     Rating_Value INT,  
5     Rating_Date DATE  
6 );
```

Table created.

4. Review Data Insertion

```
SQL> INSERT INTO Review VALUES  
2 (1, 1, 181, 'Good quality and useful storage space.', DATE '2026-09-01');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Review VALUES  
2 (2, 2, 182, 'Comfortable bed frame with a simple design.', DATE '2026-09-03');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Review VALUES  
2 (3, 3, 183, 'Nice coffee table for a small living room.', DATE '2026-09-05');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Review VALUES  
2 (4, 4, 184, 'Very comfortable and stylish armchair.', DATE '2026-09-07');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Review VALUES  
2 (5, 5, 185, 'Good desk for studying and working.', DATE '2026-09-10');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Review VALUES  
2 (6, 6, 186, 'The chair is strong and looks good.', DATE '2026-09-12');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Review VALUES  
2 (7, 7, 187, 'Simple lamp with a nice appearance.', DATE '2026-09-15');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Review VALUES  
2 (8, 8, 188, 'Easy to use and useful for organizing clothes.', DATE '2026-09-18');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Review VALUES  
2 (9, 9, 189, 'Good outdoor table with sturdy construction.', DATE '2026-09-20');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Review VALUES  
2 (10, 10, 110, 'Looks realistic and adds decoration to the room.', DATE '2026-09-22');  
  
1 row created.
```

5. Rating Data Insertion

```
SQL> INSERT INTO Rating VALUES
  2 (1, 1, 5, DATE '2026-09-01');

1 row created.

SQL>
SQL> INSERT INTO Rating VALUES
  2 (2, 2, 4, DATE '2026-09-03');

1 row created.

SQL>
SQL> INSERT INTO Rating VALUES
  2 (3, 3, 4, DATE '2026-09-05');

1 row created.

SQL>
SQL> INSERT INTO Rating VALUES
  2 (4, 4, 5, DATE '2026-09-07');

1 row created.

SQL>
SQL> INSERT INTO Rating VALUES
  2 (5, 5, 5, DATE '2026-09-10');

1 row created.

SQL>
SQL> INSERT INTO Rating VALUES
  2 (6, 6, 4, DATE '2026-09-12');

1 row created.

SQL>
SQL> INSERT INTO Rating VALUES
  2 (7, 7, 3, DATE '2026-09-15');

1 row created.

SQL>
SQL> INSERT INTO Rating VALUES
  2 (8, 8, 4, DATE '2026-09-18');

1 row created.

SQL>
SQL> INSERT INTO Rating VALUES
  2 (9, 9, 5, DATE '2026-09-20');

1 row created.

SQL>
SQL> INSERT INTO Rating VALUES
  2 (10, 10, 4, DATE '2026-09-22');

1 row created.

SQL>
SQL> COMMIT;

Commit complete.
```

6. Product Review Details

```
SQL> SELECT
  2     p.Product_ID,
  3     p.Product_Name,
  4     r.Review_ID,
  5     r.Customer_ID,
  6     r.Review_Text,
  7     r.Review_Date,
  8     rt.Rating_Value
  9 FROM Product p
 10 JOIN Review r
 11 ON p.Product_ID = r.Product_ID
 12 JOIN Rating rt
 13 ON r.Review_ID = rt.Review_ID
 14 ORDER BY p.Product_ID;
```

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_ID CUSTOMER_ID
-----
REVIEW_TEXT
-----
REVIEW_DA RATING_VALUE
-----
      101
KALLAX Shelf Unit
      1          1

PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_ID CUSTOMER_ID
-----
REVIEW_TEXT
-----
REVIEW_DA RATING_VALUE
-----
Good quality and useful storage space.
01-SEP-26          5

PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_ID CUSTOMER_ID
-----
REVIEW_TEXT
-----
REVIEW_DA RATING_VALUE
-----
      102
MALM Bed Frame
      2          2

PRODUCT_ID
-----
PRODUCT_NAME
-----
```

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_ID CUSTOMER_ID
-----
REVIEW_TEXT
-----
REVIEW_DA RATING_VALUE
-----
      110
FEJKA Artificial Plant
      10          10

PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_ID CUSTOMER_ID
-----
REVIEW_TEXT
-----
REVIEW_DA RATING_VALUE
-----
Looks realistic and adds decoration to the room.
22-SEP-26          4

10 rows selected.
```

7. Average Product Rating

This is the aggregate function part of the task.

The average rating of each product was calculated using the AVG aggregate function. The results were grouped according to the product and displayed in descending order of average rating.

```
SQL> SELECT
  2   p.Product_ID,
  3   p.Product_Name,
  4   round(avg(r.rating_value), 2) AS Average_Rating
  5 FROM Product p
  6 JOIN Review r
  7 ON p.Product_ID = r.Product_ID
  8 JOIN Rating rt
  9 ON r.Review_ID = rt.Review_ID
10 GROUP BY
11   p.Product_ID,
12   p.Product_Name
13 ORDER BY Average_Rating DESC;

PRODUCT_ID
-----
PRODUCT_NAME
-----
AVERAGE_RATING
-----
181
KILLER Shelf unit
5
185
POKER Deck
5
PRODUCT_ID
-----
PRODUCT_NAME
-----
AVERAGE_RATING
-----
189
APPLING Outdoor Table
5
191
POWERS Armchair
5
PRODUCT_ID
-----
PRODUCT_NAME
-----
AVERAGE_RATING
-----
198
PUMA Artificial Plant
4
199
PRODUCT_ID
-----
PRODUCT_NAME
-----
AVERAGE_RATING
-----
199
DINING Chair
4
199
PULPS Cotton Rack
4
PRODUCT_ID
-----
PRODUCT_NAME
-----
AVERAGE_RATING
-----
192
LACE Coffee Table
4
192
PULP Bed Frame
4
PRODUCT_ID
-----
PRODUCT_NAME
-----
AVERAGE_RATING
-----
197
ACE Table Lamp
4
19 rows selected.
```

8. Highly Rated Products

Highly rated products were identified using the HAVING clause with the average rating calculated using the AVG aggregate function. Products with an average rating of 4 or above were considered highly rated.

```
SQL> SELECT
  2     p.Product_ID,
  3     p.Product_Name,
  4     ROUND(AVG(rt.Rating_Value), 2) AS Average_Rating
  5 FROM Product p
  6 JOIN Review r
  7 ON p.Product_ID = r.Product_ID
  8 JOIN Rating rt
  9 ON r.Review_ID = rt.Review_ID
 10 GROUP BY
 11     p.Product_ID,
 12     p.Product_Name
 13 HAVING AVG(rt.Rating_Value) >= 4
 14 ORDER BY Average_Rating DESC;
```

PRODUCT_ID

PRODUCT_NAME

AVERAGE_RATING

101			
KALLAX	Shelf	Unit	5

105			
MICKE	Desk		5

PRODUCT_ID

PRODUCT_NAME

AVERAGE_RATING

PRODUCT_ID

PRODUCT_NAME

AVERAGE_RATING

103			
LACK	Coffee	Table	4

102			
MALM	Bed	Frame	4

PRODUCT_ID

PRODUCT_NAME

AVERAGE_RATING

9 rows selected.

9. Conclusion

The Product Review and Rating Management System was implemented for the IKEA E-Commerce Database Management System. Customer reviews and product ratings were stored using separate Review and Rating tables. SQL JOIN operations were used to retrieve product review details. The AVG aggregate function was used to calculate the average rating of products, and the HAVING clause was used to identify highly rated products. The implementation demonstrates the use of relational tables, joins, aggregate functions, grouping, and filtering in an e-commerce database system. What you need to prepare in Word.

